Estimation de pose et analyse cinématique : Application au geste sportif

1 Introduction

L'estimation de pose humaine (Human Pose Estimation) est une technologie d'IA permettant d'extraire les mouvements du corps à partir d'une simple vidéo. Dans le cadre de ce projet, nous appliquons cette méthode à l'analyse du coup droit d'un tennisman.

L'objectif est d'analyser le geste technique en extrayant des données biomécaniques précises et de modéliser le mouvement pour prédire la trajectoire future du bras.

2 Installation & Choix du modèle utilisé

Le choix du modèle d'estimation de pose est une étape critique qui définit la qualité des données extraites. Notre approche a évolué pour privilégier la fidélité spatiale des points clés sur la vitesse d'exécution.

Initialement, nous avons envisagé l'utilisation de RTMPose (Real-Time Multi-Person Pose Estimation). Ce modèle récent est conçu pour maximiser le débit d'inférence (FPS), ce qui aurait théoriquement permis un traitement en flux direct. Cependant, son déploiement repose sur une accélération GPU spécifique et la complexité de mise en œuvre de son environnement (dépendances strictes entre CUDA, PyTorch et la nécessité d'un GPU Nvidia disponible à la demande) nous ont conduits à réévaluer nos priorités.

Nous avons alors opté pour l'architecture HRNet (Deep High-Resolution Representation Learning). Pourquoi ce choix est-il pertinent pour notre projet ?

- **Précision Spatiale :** Pour calculer des vitesses et des accélérations (dérivées de la position), la moindre erreur de pixel est amplifiée. L'architecture de HRNet garantit une localisation des articulations plus fine que les modèles légers comme RTMPose.
- **Robustesse aux occlusions :** Le maintien des détails haute résolution permet au modèle de mieux "comprendre" la position du bras même lorsque celui-ci bouge rapidement ou passe devant le corps.
- **Traitement CPU stable :** En travaillant image par image avec HRNet sur CPU, nous sacrifions la vitesse temps réel mais nous garantissons une extraction de données déterministe et fiable, indispensable pour tracer des courbes cinématiques exploitables.

En résumé, si RTMPose est adapté à la surveillance ou au jeu vidéo, HRNet se montre comme un bon outil pour l'analyse technique et biomécanique où la justesse de la mesure prime sur l'immédiateté du résultat.

3 Tracer et relier les Keypoints

La première étape après l'installation du modèle a été de découper la vidéo source en une suite de 378 images individuelles (frames). Cette opération est réalisée via la bibliothèque OpenCV. Bien que le traitement direct en flux soit possible, nous avons opté pour une décomposition de la vidéo en frames. Ce choix technique se justifie par la nécessité de sécuriser le processus d'inférence sur CPU. En travaillant image par image, nous minimisons l'occupation de la mémoire vive et nous permettons un contrôle de la qualité précis sur chaque étape du mouvement, tout en évitant la perte de données en cas d'interruption du calcul.

Ensuite, chaque image est envoyée au modèle HRNet via MMPose. L'IA identifie les 17 points clés du corps nommés Keypoints, dont on récupère les coordonnées. Ces données sont immédiatement stockées dans un tableau, nous permettant par la suite d'effectuer des calculs cinématiques sur les points qui nous intéressent.

Une fois les points clés détectés, nous demandons à l'outil de dessiner sur chaque image. On relie les points entre eux par des segments colorés pour former le squelette cinématique. Cette étape sert avant tout à vérifier d'un coup d'œil si l'IA a bien relié tous les Keypoints ou si elle a été perturbée par le décor ou le mouvement rapide de la raquette. Chaque segment est coloré différemment pour éviter la confusion en cas de mouvement ou de rotation rapide.

La dernière étape consiste à rassembler toutes les images traitées (celles avec le squelette dessiné) pour recréer un fichier vidéo unique au format .mp4. Cette vidéo finale sert de support de présentation. Elle permet de voir en temps réel comment l'exosquelette réagit lors de la frappe et de valider la cohérence entre ce que nous voyons à l'écran et les données numériques que nous avons stockées.

4 Analyse de trajectoire

Une fois que nous avons extrait les coordonnées via HRNet, nous obtenons un fichier CSV brut. Nous avons utilisé les données du poignet droit, principal acteur du coup droit du tennisman, pour effectuer des calculs nous permettant de représenter efficacement la cinématique du mouvement.

Avant tout calcul, il est crucial de vérifier que nos données sont dans le bon ordre. Le script trie les lignes par nom d'image (frame_001, frame_002, etc.) Si les frames étaient mélangées, le calcul de la vitesse (la distance entre deux images) donnerait des résultats totalement faux.

Le premier graphique (Figure 1) représente le déplacement du poignet dans le plan 2D de l'image. En informatique, le point (0,0) d'une image est en haut à gauche. Cela signifie que plus un objet descend, plus sa coordonnée Y augmente. Pour que nos graphiques soient lisibles et à l'endroit par rapport au sol, nous avons inversé l'axe des ordonnées. De cette façon, quand le tennisman lève le bras dans la vidéo, la courbe monte aussi sur le graphique. La courbe obtenue permet de visualiser précisément la "boucle" du coup droit. On distingue nettement la phase de préparation (montée du poignet), la mise à niveau et la remontée finale lors de l'accompagnement.

Pour mesurer l'explosivité du coup droit, nous calculons la distance parcourue par le poignet entre chaque images successives. Pour cela, nous utilisons le théorème de Pythagore sur les variations de la position X et Y, cela nous donne, Vitesse $= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Ce calcul nous donne une vitesse en "pixels par frame" que nous avons représenté en fonction du temps (figure 2). Nous obtenons une vitesse moyenne de 4.84 pixels/frames, et une vitesse maximale a 137 pixels/frames, ce qui met en évidence l'intensité du coup droit.

L'IA de détection peut parfois trembler d'un pixel ou deux entre deux images, ce qui crée des pics de vitesse irréels. Pour nettoyer ces erreurs, nous avons appliqué une moyenne mobile sur 5 frames. Au lieu de regarder une seule image, on fait la moyenne avec les images voisines. La courbe devient plus lisse et beaucoup plus facile à interpréter pour identifier le geste réel du joueur sans le bruit de la machine.

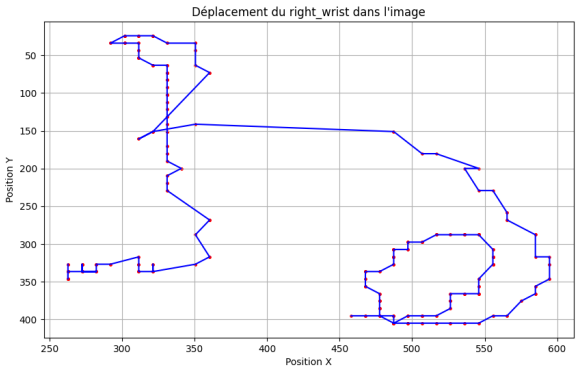

Figure 1

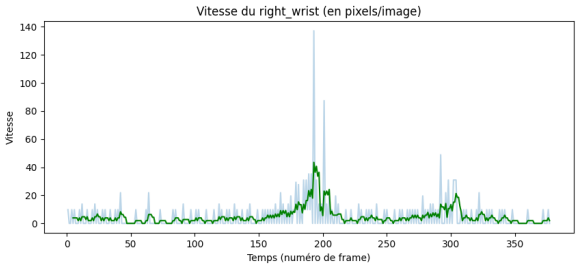

Figure 2

5 Prédiction de l'évolution du mouvement via LSTM

Enfin, le but n'était pas de se limiter à l'analyse du passé, mais de prédire le mouvement futur du joueur. Pour ce faire, nous avons implémenté un réseau de neurones LSTM (Long Short-Term Memory).

Contrairement aux réseaux classiques qui traitent les données de manière indépendante, les LSTM possèdent une "mémoire" interne leur permettant de comprendre les séquences temporelles. Ils sont donc théoriquement idéaux pour anticiper la position future d'un bras en fonction de ses positions passées.

Les données brutes issues de HRNet (fichier CSV) ne peuvent pas être injectées directement dans le réseau. Nous avons appliqué un pré-traitement en trois étapes :

1. **Filtrage :** Nous ne conservons que les coordonnées (x, y) des 17 points clés, soit 34 valeurs (features) par image.
2. **Normalisation :** Les réseaux de neurones convergent difficilement avec des valeurs de pixels allant de 0 à 1920. Nous avons utilisé un `MinMaxScaler` pour ramener toutes les coordonnées dans l'intervalle $[0, 1]$.
3. **Fenêtrage (Sliding Window) :** Pour prédire l'image $t+1$, le modèle a besoin de contexte. Nous avons structuré les données sous forme de séquences glissantes de taille N (Look Back). Ainsi, le modèle prend en entrée les frames $[t - N, ..., t]$ pour essayer de deviner la frame $t + 1$.

Nous avons conçu un modèle séquentiel avec la bibliothèque *TensorFlow/Keras* composé de :

— **Couche LSTM (128 unités) :** Avec l'option `return_sequences=True`, elle capture les motifs complexes du mouvement au fil du temps.
— **Couche LSTM (64 unités) :** Elle affine la compréhension temporelle et réduit la dimensionnalité.
— **Couche Dense (34 unités) :** La couche de sortie qui projette les résultats pour reconstituer les 34 coordonnées prédites (x, y) des articulations.

L'entraînement est réalisé avec l'optimiseur *Adam* et une fonction de perte *MSE* (Mean Squared Error), adaptée aux problèmes de régression.

Bien que l'architecture du LSTM soit fonctionnelle, nous avons été confrontés à une contrainte majeure liée à la taille du dataset. En effet, le modèle HRNet, bien que très précis, est coûteux en temps de calcul sur CPU (environ 15 minutes de traitement pour générer les points clés d'une seule vidéo de 7 secondes). Cette lenteur nous a empêchés de constituer une base de données massive de plusieurs centaines de coups droits, nécessaire pour entraîner efficacement un Deep Learning.

Avec une seule vidéo de 378 frames, notre LSTM ne dispose que de quelques centaines d'exemples d'entraînement. En conséquence, le modèle subit un phénomène d'*overfitting* (sur-apprentissage) ou tend à prédire une position moyenne statique.

Cette section constitue donc une preuve de concept. Elle démontre que le code est fonctionnel (CSV $\rightarrow$ Normalisation $\rightarrow$ LSTM $\rightarrow$ Prédiction) est opérationnel, mais qu'il nécessiterait une puissance de calcul GPU pour générer suffisamment de données d'entraînement via HRNet afin d'obtenir des prédictions fiables.

6 Pistes d'amélioration

Une piste d'évolution consisterait à intégrer l'angle du coude pour détecter précisément l'extension du bras lors de la frappe. Ce paramètre a été testé en calculant le produit scalaire entre les vecteurs coude-épaule et coude-poignet :

$$\theta = \arccos\left(\frac{\vec{BA} \cdot \vec{BC}}{\|\vec{BA}\| \times \|\vec{BC}\|}\right)$$

Cependant, cette mesure a été écartée car la vue 2D écrase la profondeur. À cause de la perspective, les angles calculés étaient souvent différents des valeurs attendus dans la réalité (figure 3). Ce biais rendait les comparaisons impossibles entre des vidéos filmées sous différents angles.

Nous avons également envisagé d'étudier la flexion des jambes, car elle constitue un signal précurseur d'une frappe, ce qui aurait permis de faciliter la prédiction du mouvement. Cependant, il y a un soucis de

distorsion des proportions lors du changements de posture. Dans cette vidéo, le tennisman est de profil et il prend une prise d'appui large, la distance en pixels entre la hanche et la cheville augmente artificiellement sur l'image à cause de l'inclinaison de la jambe. Pour l'algorithme, cela ressemble à un étirement du membre, ce qui peut fausser les calculs de vitesse et les prédictions de trajectoire qui reposent sur des coordonnées 2D.

Une piste d'amélioration serait l'utilisation d'un modèle de "3D Pose Estimation" permettant de reconstruire les geste dans l'espace et d'obtenir des mesures anatomiques exactes pour l'angle du coude, ainsi que pour la distance entre les hanches et les chevilles qui quantifie la flexion des jambes.

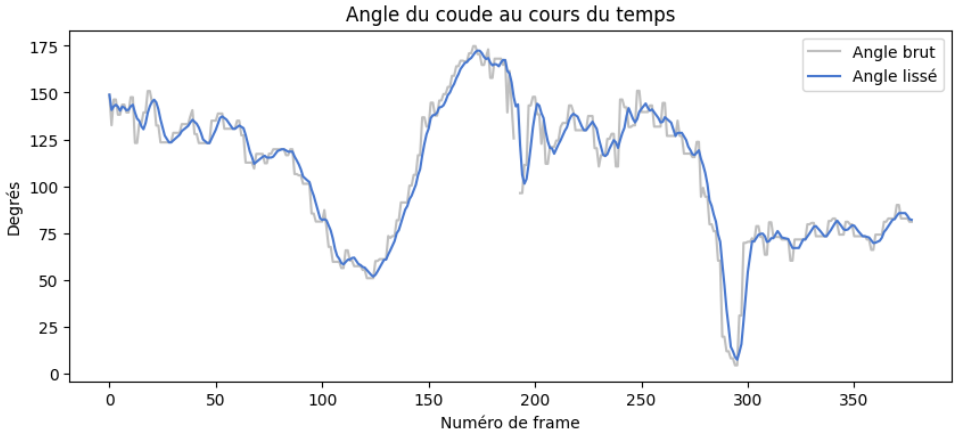

Figure 3